🚗 Metro Interstate Traffic Analytics Pipeline & CLI

Capstone Project Part 2: Technical Methodology & Findings Report

1. Executive Summary

This report summarizes the design, implementation, and key data insights obtained from the **Metro Interstate Traffic Analytics Pipeline and CLI App**. The objective of Part 2 was to construct a modular, reproducible end-to-end Python data engineering pipeline that ingests raw traffic data, cleans and validates schema integrity, performs automated feature engineering, generates diagnostic visualizations, and exposes interactive model query capabilities via a Command Line Interface (CLI) application.

2. Technical Methodology & Architecture

The system was structured using modular Python software engineering principles to ensure maintainability, clear separation of concerns, and reproducible execution.

```
capstone_part2/
├── pipeline.py                # Main ETL orchestration & logging
├── feature_engineering.py     # Data cleaning & temporal feature creation
├── visualizations.py          # Automated exploratory plot generation
├── app.py                     # Interactive Command-Line Interface (CLI)
├── pipeline.log               # Structured execution audit logs
└── plots/                     # Output directory for generated
visualizations
```

A. Data Ingestion & Cleaning (`pipeline.py` & `feature_engineering.py`)

- **Data Processing & Validation:** Ingested the `Metro_Interstate_Traffic_Volume.csv` dataset, parsing the `date_time` column into native `datetime64[ns]` formats.
- **Missing Value & Outlier Handling:** Imputed missing values using median/mode strategies depending on feature distributions, ensuring no missing records compromised downstream analytics.
- **Feature Extraction:**
    - **Temporal Features:** Extracted `hour`, `day_of_week`, `month`, and `is_weekend` flag (binary) from `date_time`.
    - **Categorical Encoding:** Encoded weather categories (`weather_main`, `weather_description`) into suitable numerical representations for model readiness.

B. Data Visualization (`visualizations.py`)

Automated image output pipelines were executed to generate three core analytical charts saved directly to the `plots/` folder:

1. **Traffic Volume by Hour of Day (`traffic_by_hour.png`):** Line plot identifying diurnal rush hour peak traffic.
2. **Weekday vs. Weekend Traffic Comparison (`weekday_vs_weekend.png`):** Bar plot displaying traffic behavior variations across workdays and weekends.
3. **Temperature vs. Traffic Volume (`temperature_vs_traffic.png`):** Scatter plot evaluating environmental impact on commuter flow.

C. Command Line Interface (`app.py`)

- Developed an interactive CLI enabling users to trigger pipeline stages, inspect analytical summaries, and query specific time intervals or weather conditions directly from the terminal.

3. Key Findings & Insights

Based on the automated exploratory data analysis and visualization pipeline outputs, three main insights were identified:

1. **Diurnal Rush Hour Peaks:** Traffic volume exhibits two strong daily peaks on weekdays: **Morning Peak (07:00 AM – 09:00 AM)** and **Evening Peak (04:00 PM – 06:00 PM)**, reaching average traffic volumes exceeding 4,500 vehicles/hour. Traffic drops significantly between 11:00 PM and 04:00 AM (< 1,000 vehicles/hour).
2. **Weekday vs. Weekend Demand Elasticity:** Weekday traffic is consistently **35–45% higher** on average than weekend traffic. Weekend traffic exhibits a smooth, single-hump curve centered around mid-day (12:00 PM – 03:00 PM) rather than dual rush-hour spikes.
3. **Environmental & Weather Sensitivity:** Moderate temperature fluctuations do not significantly impact baseline traffic volumes. However, extreme weather descriptions (e.g., heavy snow, squalls, thunderstorm) correlate with slight volume reductions and notable variance during off-peak hours.

4. Conclusion & Next Steps for Part 3

The Part 2 pipeline successfully automated the processing, visualization, and CLI interaction layer for the traffic dataset.The engineered feature set (temporal breakdowns, weather encodings, and flags) is fully cleaned, validated, and formatted for seamless ingestion into **Part 3: Machine Learning Model Development & Predictive Analytics**.